Higher-Order Components (HOC)

3 min read Updated 24 Mar 2026 Topic 31 of 62

6.2 - Higher-Order Components (HOC)

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"What is a Higher-Order Component?"	🟡🔴	★★★★☆
"Give a real-world example of an HOC"	🟡🔴	★★★★☆
"What are the drawbacks of HOCs?"	🔴	★★★☆☆
"HOC vs Custom Hooks: which do you prefer?"	🟡🔴	★★★★☆

The Pattern

A Higher-Order Component is a function that takes a component and returns a new component with enhanced behavior. It's a pattern, not a React API. The concept comes

JSX Copy

```jsx
// The HOC
function withAuth(WrappedComponent) {
  return function AuthenticatedComponent(props) {
    const { user } = useAuth();

    if (!user) {
      return <Navigate to="/login" />;
    }

    return <WrappedComponent {...props} user={user} />;
  };
}

// Usage: wrap any component to add auth protection
const ProtectedDashboard = withAuth(Dashboard);
const ProtectedSettings = withAuth(Settings);
```

The HOC doesn't modify the original component. It wraps it in a new component that adds functionality (in this case, authentication checking).

Real-World HOC Examples

withLoading: Shows a spinner while data loads

JSX Copy

```jsx
function withLoading(WrappedComponent) {
  return function LoadingComponent({ isLoading, ...props }) {
    if (isLoading) return <Spinner />;
    return <WrappedComponent {...props} />;
  };
}

const UserListWithLoading = withLoading(UserList);

// Usage
<UserListWithLoading isLoading={loading} users={users} />
```

JSX Copy

```jsx
function withStyles(WrappedComponent) {
  return function StyledComponent(props) {
    const style = { padding: '16px', margin: '8px', borderRadius: '8px' };
    return <WrappedComponent style={style} {...props} />;
  };
}
```

Drawbacks of HOCs

1. **Wrapper hell.** Nesting multiple HOCs creates deeply nested component trees that are hard to debug: `withAuth(withLoading(withStyles(MyComponent)))`.
2. **Prop collisions.** If two HOCs inject a prop with the same name, one overwrites the other silently.
3. **Difficult to trace.** In React DevTools, you see layers of anonymous wrapper components instead of your actual component.
4. **Static composition only.** You can't use HOCs inside a render method or conditionally. They must be applied at the module level.

HOC vs Custom Hooks

Custom hooks solve the same problem (sharing logic between components) without the drawbacks. In modern React, hooks are the preferred approach.

Feature	HOC	Custom Hook
How it works	Wraps the component	Called inside the component
Wrapper hell	Yes, nesting gets messy	No wrappers, flat structure

DevTools readability	Poor (anonymous wrappers)	Clean (hooks show as hooks)
Conditional usage	Cannot use conditionally	Same restriction (hooks rules apply)
Modern preference	Legacy pattern	Preferred approach

JSX Copy

```jsx
// HOC approach (legacy)
const EnhancedComponent = withAuth(withLoading(MyComponent));

// Custom hook approach (modern)
function MyComponent() {
  const { user } = useAuth();
  const { data, loading } = useFetch('/api/data');

  if (!user) return <Navigate to="/login" />;
  if (loading) return <Spinner />;
  return <div>{data}</div>;
}
```

Interview answer: "HOCs are functions that take a component and return a new one with added behavior. They were the primary way to share logic before hooks. Real-world examples include withAuth for route protection and withLoading for loading states. The main drawbacks are wrapper hell, prop collisions, and poor DevTools readability. In modern React, I prefer custom hooks because they achieve the same code reuse without wrapping. But I understand HOCs because many codebases still use them, and libraries like React Redux's `connect()` are HOCs."

QUICK CHECK — no marks, just for you.

1. According to the lesson, what most accurately describes a Higher-Order Component?

- ◯ A pattern: a function that takes a component and returns a new component with enhanced behavior
- ◯ A component that renders other components passed to it through the children prop
- ◯ A built-in React hook that lets one component reuse another component's state
- ◯ A React API method that mutates a component class to add lifecycle behavior

Check answer

2. You compose two HOCs and both inject a prop called user into the wrapped component. Based on the lesson, what happens?

- ◯ React logs a development warning and keeps the outermost HOC's value
- ◯ The wrapped component receives an array containing both values
- ◯ One prop silently overwrites the other
- ◯ React throws a duplicate prop error at render time

Check answer

3. A candidate argues: "Custom hooks are better than HOCs because, unlike HOCs, you can apply a hook conditionally inside a render branch." Per the lesson's HOC vs custom hooks comparison, what is wrong with this reasoning?

Hooks carry the same restriction, since the rules of hooks forbid conditional calls; the real hook advantages are no wrapper hell, no prop collisions, and cleaner DevTools

- ○ Nothing is wrong; conditional usage is exactly the advantage hooks have over HOCs
- ○ It is wrong because HOCs have no restriction at all; they can be composed anywhere, including inside render
- ○ It is backwards: HOCs can be applied conditionally at render time, while hooks cannot

Check answer

‹ Previous
Container/Presentational Pattern

Next ›
Render Props Pattern

LEARN MORE

ARTICLES & DOCS

Higher-Order Components (docs)
reactjs.org

VIDEOS



HOC in 30 minutes for beginners
YouTube



Learn HOC in 10 minutes
YouTube



HOC as a design pattern
YouTube

Hand-picked references for this lesson.



React.js Interview Kit